Weekly Review
R110 - Boolean Algebra and File Sizes

Complete the weekly review, you'll have
15 minutes.

Weekly Review
R110 - Boolean Algebra and File Sizes

Mark that review!

Once you've done it:

1. Submit the score into the Google Form.
2. Update your weekly review tracker with **targets**.

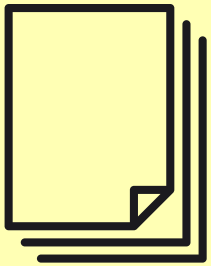
Starter

Define the following terms:

- Machine code
- Instruction Set
- Operation Code
- Operand
- Register

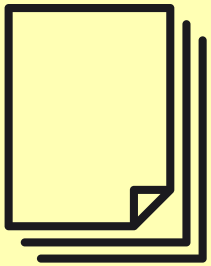
Assembly Language

Processor Instructions



At the processor level, instructions do not look like C# or SQL (or even Python) code.

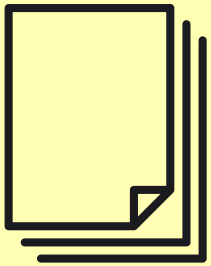
Processor Instructions



At the processor level, instructions do not look like C# or SQL (or even Python) code.

Instead, they are in a simpler format called **machine code** (just binary).

Processor Instructions



At the processor level, instructions do not look like C# or SQL (or even Python) code.

Instead, they are in a simpler format called **machine code** (just binary).

Machine code can be easily interpreted in **assembly form** for human readability.

Processor Instructions

Binary Form `0001 0001 0010 1111`

Assembly Form `ADD R1, R2, #15`

Processor Instructions

Binary Form `0001 0001 0010 1111`

Assembly Form `ADD R1, R2, #15`

Operation Code	Operand	Operand	Operand
ADD	R1	R2	#15

Processor Instructions

Binary Form `0001 0001 0010 1111`

Assembly Form `ADD R1, R2, #15`

Operation Code	Operand	Operand	Operand
ADD	R1	R2	#15

Instruction set

The set of binary codes for the machine operations that a processor has been designed to perform
(e.g. ADD, LOAD, SUBTRACT, LEFT SHIFT)

OpCode

These are short abbreviations for what the instruction does.

Called mnemonics

OpCode

These are short abbreviations for what the instruction does.

Called mnemonics

Generally between 1-4 letters

OpCode

These are short abbreviations for what the instruction does.

Called mnemonics

Generally between 1-4 letters

Eg ADD, AND, B, BNE, HALT

Operand

The “data” or “inputs” that the instruction uses.

Operand

The “data” or “inputs” that the instruction uses.

A bit like procedure parameters

Operand

The “data” or “inputs” that the instruction uses.

A bit like procedure parameters

Eg ADD R1,R2,R3

Operand

The “data” or “inputs” that the instruction uses.

A bit like procedure parameters

Eg ADD R1,R2,R3

Rx refers to general purpose register x (on the CPU)

Operand

The “data” or “inputs” that the instruction uses.

A bit like procedure parameters

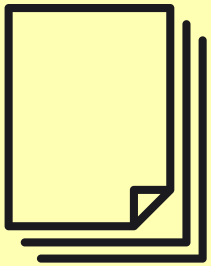
Eg ADD R1,R2,R3

Rx refers to general purpose register x (on the CPU)

There are three addressing modes:

- **Immediate Addressing** (#15)
- **Direct (register) Addressing** (R1)
- **Direct (memory) Addressing** (15)

How does it go to code?



Assembly code instructions are **more human friendly** than machine code instructions.

They are converted by an **assembler** into machine code that can be run on a computer.

This process occurs as a set of **one-to-one** mappings.

What can Assembly do?

Very little per instruction!

- Move data around (between CPU registers and main memory)

What can Assembly do?

Very little per instruction!

- Move data around (between CPU registers and main memory)
- Perform simple arithmetic and logical operations on data in registers (eg add, subtract, and)

What can Assembly do?

Very little per instruction!

- Move data around (between CPU registers and main memory)
- Perform simple arithmetic and logical operations on data in registers (eg add, subtract, and)
- Simple branching (jumping to a different instruction in the code). Can be conditional (like an IF statement, but far less complex).

What can Assembly do?

Very little per instruction!

- Move data around (between CPU registers and main memory)
- Perform simple arithmetic and logical operations on data in registers (eg add, subtract, and)
- Simple branching (jumping to a different instruction in the code). Can be conditional (like an IF statement, but far less complex).
- A few other instructions (eg HALT)

AQA Instruction Set

You don't need to remember them!

"In the exam, students will be provided with a list of assembly language instructions so they will not need to recall the instruction set."

You will, of course, need to know what they do and how to use them.

You have a copy of the instruction set on Classroom
(**L142 – Assembly Instruction Set**)

In an exam, you will be given them all if you need.

Examples

```
MOV R1, #45
```

Examples

```
MOV R1, #45
```

- puts the immediate value 45 into register R1

Examples

```
MOV R1, #45
```

- puts the immediate value 45 into register R1

```
LDR R1, 45
```

Examples

```
MOV R1, #45
```

- puts the immediate value 45 into register R1

```
LDR R1, 45
```

- Puts the contents of memory location 45 into R1

Examples

```
MOV R1, #45
```

- puts the immediate value 45 into register R1

```
LDR R1, 45
```

- Puts the contents of memory location 45 into R1
- A Direct (memory) addressing instruction will take longer to execute since the main memory will need to be accessed (involving MAR, MBR, buses etc.)

Example Programs

```
MOV R1, #23
ADD R1, R1, #41
HALT
```

What is the effect of this program when run?

Example Programs

```
MOV R1, #23
ADD R1, R1, #41
HALT
```

What is the effect of this program when run?

Gives 64 in R1

Example Programs

```
LDR R1, 45
LDR R2, 46
SUB R1, R1, R2
STR R1, 47
HALT
```

Address	Value
45	36
46	31
47	

What about this program?

Example Programs

```
LDR R1, 45
LDR R2, 46
SUB R1, R1, R2
STR R1, 47
HALT
```

Address	Value
45	36
46	31
47	

What about this program?

This will give 5 at address 47

If Statements

```
MOV R1, #5
CMP R1, #6
BNE notEqualPart
<code>
B endif
notEqualPart:
<more code>
endif:
```

Practice

Write programs to achieve:

1. A 23 & 12, storing the answer in R6
2. Add 54 and 77, storing the answer at memory location 9

Assembly Practice

https://compsci.barton.ac.uk/csStudent/assemblySimulator/

Play around and then start the worksheet:

W120 - Assembly Language Problems

Checking Question

What does this do?

```
MOV R1, #5
start:
<code>
SUB R1, R1, #1
CMP R1, #0
BNE start
HALT
```

Checking Question

What does this do?

```
MOV R1, #5
start:
<code>
SUB R1, R1, #1
CMP R1, #0
BNE start
HALT
```

Simple loop